

to do is generate a list of every possible move in our game. We can do this with a couple of nested loops that consider every possible connection between points in our lattice:

```
allpossible = []

gamesize = 5

for i in range(1,gamesize + 1):
    for j in range(2,gamesize + 1):
        allpossible.append([(i,j),(i,j - 1)])

for i in range(1,gamesize):
    for j in range(1,gamesize + 1):
        allpossible.append([(i,j),(i + 1,j)])
```

This snippet starts by defining an empty list, called `allpossible`, and a `gamesize` variable, which is the length of each side of our lattice. Then, we have two loops. The first is meant to add vertical moves to our list of possible moves. Notice that for every possible value of `i` and `j`, this first loop appends the move represented by `[(i,j),(i,j - 1)]` to our list of possible moves. This will always be a vertical line. Our second loop is similar, but for every possible combination of `i` and `j`, it appends the horizontal move `[(i,j),(i + 1,j)]` to our list of possible moves. At the end, our `allpossible` list will be populated with every possible move.

If you think about a game that's in progress, like the game illustrated in [Figure 10-2](#), you'll realize that not every move is always possible. If a player has already played a particular move during a game, no player can play that same move again for the rest of the game. We'll need a way to remove all moves that have already been played from the list of all possible moves, resulting in a list of all possible moves remaining for any particular in-progress game. This is easy enough:

```
for move in allpossible:
    if move in game:
        allpossible.remove(move)
```

As you can see, we iterate over every move in our list of possible moves, and if it's already been played, we remove it from our list. In the end, we